EECE 7205: Fundamentals of Computer Engineering

Fall 2026

In-Class Work: Deleting from a Binary Search Tree

Student Name: Natan Blumberg
Instructor: Dr. Fatema Nafa

1 Case 1: Deleting the Root (case1.cpp)

Drawings: Figures 1 and 2. Runs: Figure 3 (`case1`, `case1_q2`, `case1_q3`, `case1_q4`).

1.1 Question 1

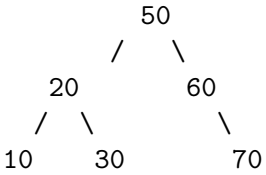
The search starts with `parent = root` (40) and `succ = root->right` (60). Each loop test checks `succ->left`.

Step	`parent`	`succ`	`succ->left`
Start	40	60	50, so keep going left
After one loop	60	50	NULL, so stop

The nodes visited are 40 (the start), 60 and 50. The successor is 50 and its parent is 60.

1.2 Question 2

50 is copied into the root, and the old leaf 50 is removed by setting `60->left` to NULL.



Inorder: 10 20 30 50 60 70
Preorder: 50 20 10 30 60 70

Rule B7 still holds because the inorder traversal is strictly increasing. Inorder visits the left subtree, then the node, then the right subtree, so the output can only come out sorted if every node is larger than everything on its left and smaller than everything on its right, which is the BST property.

To print the preorder, `case1_q2.cpp` adds one function and three lines after the After line:

```
// Preorder (Node, Left, Right) prints the root first
void printPreorder(Node* node)
{
    if (node == nullptr) return;
    cout << node->data << " ";
    printPreorder(node->left);
    printPreorder(node->right);
}

    cout << "Preorder: ";
    printPreorder(root);
    cout << endl;
```

Output:

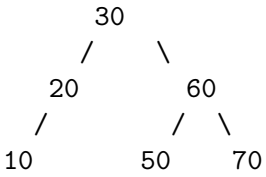
```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 50 60 70   (root = 50)
Preorder: 50 20 10 30 60 70
```

1.3 Question 3

`case1_q3.cpp` replaces steps 1 to 3 with the predecessor version:

```
// 1. find the inorder predecessor: the largest value in the left subtree
Node* parent = root;
Node* pred = root->left;               // go left once (20)
while (pred->right != nullptr)         // then right as far as possible
{
    parent = pred;
    pred = pred->right;                // pred = 30, parent = 20
}
// 2. copy the predecessor's value into the root
root->data = pred->data;               // root now holds 30
// 3. remove the predecessor node (it has no right child)
if (parent == root)
    parent->left = pred->left;
else
    parent->right = pred->left;        // 20->right = NULL
delete pred;
```

The search goes left once to 20. 20 has a right child, so `pred` moves to 30 with `parent` = 20. 30 has no right child, so the loop stops. 30 is copied into the root, and because `parent` is not the root, `20->right` is set to `pred->left`, which is NULL.



Output:

```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 50 60 70   (root = 30)
```

The new root is 30. The inorder output does not change: it is 10 20 30 50 60 70 in both versions. Inorder prints the values of a BST in sorted order, and both versions delete the same key and keep the same six values, so the sorted list is the same. Only the shape of the tree changes, which shows up in the root (30 instead of 50).

1.4 Question 4

In tree P the search goes right once to 65, then left to 55. 55 has no left child, so the successor is 55 and its parent is 65. 55 is copied into the root.

The successor node is removed with rule B4, not B3, because it is not a leaf: 55 has a right child, 60. Setting `65->left` to NULL as B3 does would cut 60 out of the tree. Instead 60 takes 55's place. The code's else branch runs `parent->left = succ->right`, which sets `65->left` to 60.

```
           55
         /    \
   25           65
  /   \        /  \
15     35    60    75
        /
      30
```

Output:

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 25 30 35 55 60 65 75   (root = 55)
```

`case1_q4.cpp` keeps the deletion code and only replaces tree T with tree P. The same block builds tree P in every program that uses it:

```
// Build tree P (insert 45, 25, 65, 15, 35, 55, 75, 30, 60)
Node* root = new Node(45);
root->left = new Node(25);
root->right = new Node(65);
root->left->left = new Node(15);
root->left->right = new Node(35);
root->right->left = new Node(55);
root->right->right = new Node(75);
root->left->right->left = new Node(30);
root->right->left->right = new Node(60);
```

2 Case 2: Deleting a Leaf (case2.cpp)

Drawings: Figures 4 and 5. Runs: Figure 6 (`case2`, `case2_q2`, `case2_q3`).

2.1 Question 1

Node visited	Comparison	Direction
40	$10 < 40$	left
20	$10 < 20$	left
10	$10 = 10$	found, the loop stops

The parent is 20, and 10 is its left child, so `parent->left == cur` is true and `20->left` is set to NULL.

```
      40
    /    \
20          60
  \        /  \
   30    50    70
```

Output of `case2.cpp`:

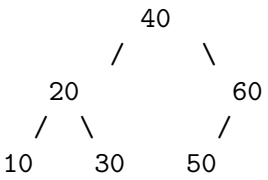
```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  20 30 40 50 60 70   (root = 40)
```

2.2 Question 2

Changed line in `case2_q2.cpp`:

```
int key = 70;
```

The search goes right at 40 ($70 > 40$) and right at 60 ($70 > 60$), and finds 70 with parent 60. `60->left` is 50, not `cur`, so the else branch runs and `60->right` is set to NULL.



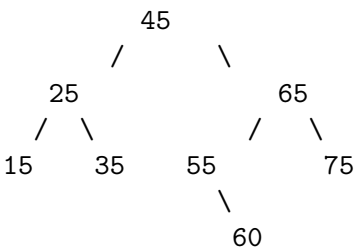
Output:

```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 40 50 60   (root = 40)
```

2.3 Question 3

The leaves of tree P are 15, 30, 60 and 75.

`case2_q3.cpp` builds tree P and sets `key = 30`. The search goes left at 45 ($30 < 45$), right at 25 ($30 > 25$) and left at 35 ($30 < 35$), and finds 30 with parent 35. 30 is 35's left child, so `35->left` is set to NULL. 35 has just become a leaf.



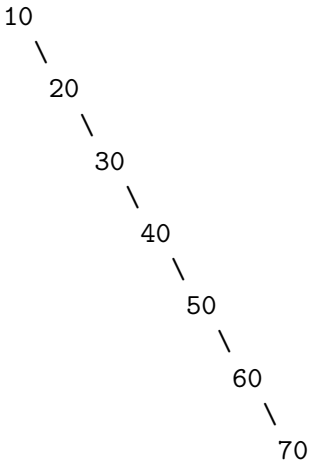
Output:

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 25 35 45 55 60 65 75   (root = 45)
```

2.4 Question 4

The search compares the key with one node on each level, from the root on level 0 down to level h, so the worst case is $h + 1$ comparisons. Counted as C++ tests, the loop checks `cur->data != key` at all $h+1$ nodes and `key < cur->data` at the first h, which is $2h+1$ tests. Both counts grow linearly with h.

For tree T, $h = 2$, so deleting a leaf takes at most 3 comparisons (for example 10: 40, 20, 10). Inserting 10, 20, 30, 40, 50, 60, 70 in that order puts each new value to the right of the one before, which gives a chain with $h = 6$. Deleting the leaf 70 then takes 7 comparisons.



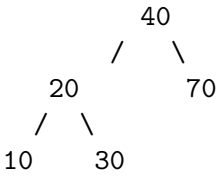
Removing a leaf is a single pointer change, so the time to delete a leaf is the time to find it, $O(h)$. When the tree is balanced, h is about $\log_2 n$ and deletion is fast. When the values are inserted in sorted order, $h = n - 1$ and deletion is as slow as walking a linked list, $O(n)$.

3 Case 3: Deleting a Node with One Child (case3.cpp)

Drawings: Figures 7 and 8. Runs: Figure 9 (`case3`, `case3_q2`, `case3_q3`, `case3_q4a`, `case3_q4b`).

3.1 Program Run

`case3.cpp` first removes 50 from tree T and then deletes 60. 60's only child, 70, moves up. 60 is 40's right child, so `40->right` is set to 70.



Output:

```
Before: 10 20 30 40 50 60 70   (root = 40)
Middle: 10 20 30 40 60 70  (60 now has one child, 70)
After:  10 20 30 40 70   (root = 40)
```

3.2 Question 1

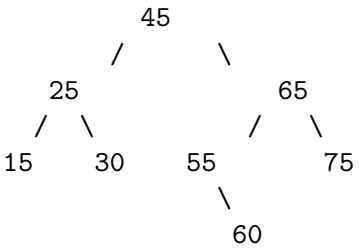
Two nodes of tree P have exactly one child: 35, whose child 30 is a left child, and 55, whose child 60 is a right child. 45, 25 and 65 have two children, and 15, 30, 60 and 75 are leaves.

3.3 Question 2

`case3_q2.cpp` builds tree P, removes the lines that delete 50 and print the Middle line (they only set up tree T), and sets the key:

```
int key = 35;
```

The search goes left at 45 ($35 < 45$) and right at 25 ($35 > 25$), and finds 35. The parent is 25 and the child that moves up is 30. 35 is 25's right child, so the else branch runs and the pointer that changes is `25->right`, from 35 to 30.

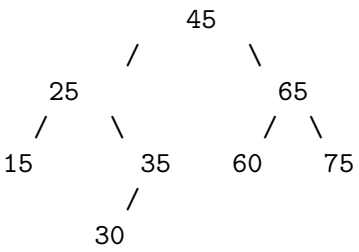


Inorder: 15 25 30 45 55 60 65 75
Output:

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 25 30 45 55 60 65 75   (root = 45)
```

3.4 Question 3

`case3_q3.cpp` is the same program with `key = 55`. The search goes right at 45 ($55 > 45$) and left at 65 ($55 < 65$), and finds 55. The parent is 65 and the child that moves up is 60. The pointer that changes is `65->left`, from 55 to 60.



Inorder: 15 25 30 35 45 60 65 75
Output:

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 25 30 35 45 60 65 75   (root = 45)
```

The program uses `parent->left` this time because the if statement checks which side of the parent the deleted node hangs on, not which side its child is on. 55 is smaller than 65, so it is 65's left child and `parent->left == cur` is true. In Question 2, 35 was larger than 25, so it was 25's right child. 60 was a right child of 55 but now sits in a left link, which is safe because everything below 55 is smaller than 65.

3.5 Question 4

Each value inserted is larger than the one before, so 10, 20, 30 form a chain going right.

```
Before (insert 10, 20, 30)      After deleting 10
```



Rule B6 says the root pointer itself must change, because the root has only one child. The new root is 20.

If `case3.cpp` is used unchanged, the search finds 10 at the root on the very first test, so the loop body never runs and `parent` stays `nullptr`. The next line evaluates `parent->left`, which dereferences a null pointer. `case3_q4a.cpp` (new tree and key, deletion code unchanged) prints the Before line and then crashes with a segmentation fault. Even without the crash, nothing in the code updates `root`, so it would still point to the freed node.

The fix in `case3_q4b.cpp` adds a branch for the root:

```
if (parent == nullptr)        root = child;  // new root is 20
else if (parent->left == cur) parent->left = child;
else                          parent->right = child;
```

Output of `case3_q4b.cpp`:

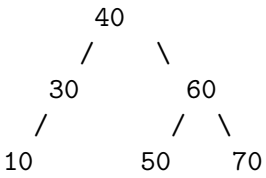
```
Before: 10 20 30   (root = 10)
After:  20 30   (root = 20)
```

4 Case 4: Deleting a Node with Two Children (case4.cpp)

Drawing: Figure 10. Runs: Figure 11 (`case4`, `case4_q1`, `case4_q3`, `case4_q4`).

4.1 Program Run

To delete 20 in tree T, the successor is 20's right child, 30. 30 has no left child, so the loop does not run and `parent` stays at `cur`. 30 is copied into the node, and `parent->right = succ->right` sets the node's right link to NULL.



Output:

```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 30 40 50 60 70   (root = 40)
```

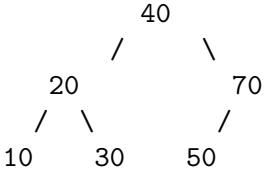
4.2 Question 1

Changed line in `case4_q1.cpp`:

```
int key = 60;
```

The search goes right at 40 and finds 60. `succ` starts at 60's right child, 70, which has no left child, so the loop does not run. After the loop, `succ` is 70 and `parent` is 60 (the same node as `cur`).

The if branch (`parent == cur`) runs. The successor is `cur`'s own right child, so the link that points to it is `cur->right`, not a left link. 70 is copied into the node, and its right link is set to `succ->right`, which is NULL.



Output:

```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 40 50 70   (root = 40)
```

4.3 Question 2

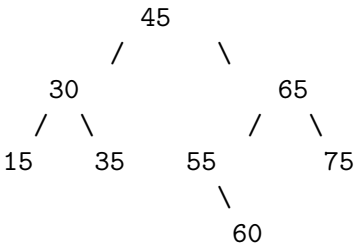
The successor is the smallest value in the node's right subtree. If it had a left child, that child would be smaller and still inside the same right subtree, so the successor would not have been the smallest after all. The code guarantees the same thing, because the loop only stops when `succ->left` is `nullptr`.

Step 4 only reconnects `succ->right`. If the successor had a left subtree, deleting the successor would cut that subtree off: its values would disappear from the tree and its nodes would never be freed.

4.4 Question 3

`case4_q3.cpp` builds tree P and sets `key = 25`. The search goes left at 45 and finds 25. The successor search goes right once to 35. 35 has a left child, so it moves left to 30 with `parent` = 35. 30 has no left child, so the successor is 30 and its parent is 35.

30 is copied into the node. `parent` is not `cur`, so the else branch runs, and the pointer that changes is `35->left`, from 30 to `succ->right`, which is NULL.



Inorder: 15 30 35 45 55 60 65 75

Output:

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 30 35 45 55 60 65 75   (root = 45)
```

4.5 Question 4

`case4_q4.cpp` replaces steps 2 to 4 with the predecessor version:

```
// 2. find the inorder predecessor: left once, then right as far as possible
Node* parent = cur;
Node* pred = cur->left;                // 15
while (pred->right != nullptr)
{
    parent = pred;
    pred = pred->right;
}
// 3. copy the predecessor's value into the node
cur->data = pred->data;                // the node that held 25 now holds 15
// 4. remove the predecessor node (it has no right child)
if (parent == cur)
    parent->left = pred->left;         // 15 was cur's left child
else
    parent->right = pred->left;
delete pred;
```

The predecessor search goes left once to 15. 15 has no right child, so the predecessor is 15 and its parent is the node being deleted. 15 is copied into the node, and `parent->left = pred->left` sets the node's left link to NULL.

```
Question 3 (successor)       Question 4 (predecessor)

         45                           45
       /    \                       /    \
   30          65               15          65
  /  \        /  \                 \        /  \
15    35    55    75                 35    55    75
            \                       /    \
             60                   30      60
```

Output:

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 30 35 45 55 60 65 75   (root = 45)
```

It is not the same tree as in Question 3. In Question 3 the node holds 30 and has two children, 15 and 35. In Question 4 it holds 15 and has only a right child, 35, with 30 still below 35. Both are valid binary search trees: their inorder traversals are the same, 15 30 35 45 55 60 65 75, and strictly increasing (rule B7).

5 Case 5: The Key Is Not in the Tree (case5.cpp)

Drawing: Figure 12. Runs: Figure 13 (`case5`, `case5_q2`, `case5_q3`, `case5_q4a`, `case5_q4b`).

5.1 Question 1

The search for 25 goes left at 40 ($25 < 40$), right at 20 ($25 > 20$) and left at 30 ($25 < 30$), where it reaches NULL. It makes 3 comparisons and stops at 30's left link, `30->left`.

Output of `case5.cpp`:

```
Before: 10 20 30 40 50 60 70   (root = 40)
25 is not in the tree: nothing to delete
After:  10 20 30 40 50 60 70   (root = 40)
```

5.2 Question 2

Key	Path (L = left, R = right)	Comparisons	Stops at
25	40 L, 20 R, 30 L	3	`30->left`
5	40 L, 20 L, 10 L	3	`10->left`
45	40 R, 60 L, 50 L	3	`50->left`
75	40 R, 60 R, 70 R	3	`70->right`

None of them needs more comparisons than the others: all four need 3, which is $h + 1$. A missing key always ends at a NULL link below a node, and every leaf of tree T is at depth $h = 2$, so every unsuccessful search in T costs exactly $h + 1$ comparisons. In any tree, the most expensive unsuccessful search costs $h + 1$.

`case5_q2.cpp` counts the comparisons for all four keys in one run:

```
int keys[] = {25, 5, 45, 75};
for (int i = 0; i < 4; i++)
{
    Node* parent = nullptr;
    Node* cur = root;
    int key = keys[i];
    int comparisons = 0;
    while (cur != nullptr && cur->data != key)
    {
        comparisons = comparisons + 1;
        parent = cur;
        if (key < cur->data) cur = cur->left;
        else                 cur = cur->right;
    }
    if (cur == nullptr)
    {
        cout << key << ": " << comparisons << " comparisons, stopped at ";
        if (key < parent->data) cout << parent->data << "->left" << endl;
        else                    cout << parent->data << "->right" << endl;
    }
}
```

Output:

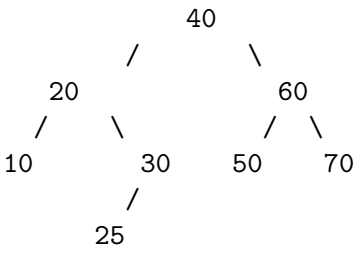
```
Before: 10 20 30 40 50 60 70   (root = 40)
25: 3 comparisons, stopped at 30->left
5: 3 comparisons, stopped at 10->left
45: 3 comparisons, stopped at 50->left
75: 3 comparisons, stopped at 70->right
After:  10 20 30 40 50 60 70   (root = 40)
```

5.3 Question 3

`case5_q3.cpp` remembers the parent during the search and inserts the key at the NULL link where the search stopped:

```
Node* parent = nullptr;
Node* cur = root;
int key = 25;
while (cur != nullptr && cur->data != key)
{
    parent = cur;
    if (key < cur->data) cur = cur->left;    // 25 < 40: go left to 20
    else                 cur = cur->right;   // 25 > 20: go right to 30
}                                            // 25 < 30: go left, NULL
if (cur == nullptr)
{
    Node* newNode = new Node(key);
    if (parent == nullptr)          root = newNode;
    else if (key < parent->data)    parent->left = newNode;  // 30->left
    else                            parent->right = newNode;
    cout << key << " was not in the tree: inserted it" << endl;
}
```

25 becomes the left child of 30.



Output:

```
Before: 10 20 30 40 50 60 70   (root = 40)
25 was not in the tree: inserted it
After:  10 20 25 30 40 50 60 70   (root = 40)
```

5.4 Question 4

With `root = nullptr`, the search loop is safe because it tests `cur != nullptr` before reading `cur->data`. The problem is the Before line, which prints `root->data`. That dereferences a null pointer, so `case5_q4a.cpp` crashes with a segmentation fault before the Before line is complete.

`case5_q4b.cpp` checks for an empty tree in the Before and After prints:

```
cout << "Before: ";
if (root == nullptr)
    cout << "the tree is empty" << endl;
else
{
    printInorder(root);
    cout << "  (root = " << root->data << ")" << endl;
}
```

The After print uses the same check. Output:

```
Before: the tree is empty
25 is not in the tree: nothing to delete
After:  the tree is empty
```

Rule B2 covers this case. The search starts at the root, which is already NULL, so it reaches NULL straight away: the key is not in the tree and nothing changes.

6 Bonus: One Function for Every Case

Drawing: Figure 14. Run of `bonus.cpp`: Figure 15.

```
Node* deleteNode(Node* root, int key)
{
    Node* parent = nullptr;
    Node* cur = root;
    while (cur != nullptr && cur->data != key)
    {
        parent = cur;
        if (key < cur->data) cur = cur->left;
        else                 cur = cur->right;
    }

    if (cur == nullptr)
    {
        cout << key << " is not in the tree: nothing to delete" << endl;
        return root;
    }

    if (cur->left != nullptr && cur->right != nullptr)
    {
        Node* succParent = cur;
        Node* succ = cur->right;
        while (succ->left != nullptr)
        {
            succParent = succ;
            succ = succ->left;
        }
        cur->data = succ->data;
        if (succParent == cur) succParent->right = succ->right;
        else                   succParent->left = succ->right;
        delete succ;
        return root;
    }

    Node* child = (cur->left != nullptr) ? cur->left : cur->right;
    if (parent == nullptr)        root = child;
    else if (parent->left == cur) parent->left = child;
    else                          parent->right = child;
    delete cur;
    return root;
}
```

Case	Rule	How `deleteNode` handles it
Search	B1	walks down from the root, remembering the parent
Not found	B2	`cur` is `nullptr`: returns the root unchanged
Two children	B5	copies the successor's value, then unlinks the successor
One child or leaf	B4, B3	the parent's link takes `child`, which is NULL for a leaf
Root with 0 or 1 child	B6	`parent` is `nullptr`, so `child` becomes the new root

The test deletes 40, 10, 60 and 25 from tree T, one after another:

```
int keys[] = {40, 10, 60, 25};
for (int i = 0; i < 4; i++)
{
    root = deleteNode(root, keys[i]);
    cout << "After deleting " << keys[i] << ": ";
    printInorder(root);
    cout << "  (root = " << root->data << ")" << endl;
}
```

The four deletions cover every case. 40 is the root with two children, 10 is a leaf, and 60 has one child because the first deletion removed 50 (40's successor). 25 is not in the tree.

```
After deleting 40    After deleting 10    After deleting 60 (25: no change)

         50                  50                    50
       /   \               /   \                 /   \
   20        60        20        60          20        70
  /  \         \        \         \           \
10    30        70       30        70          30
```

Output:

```
Before:             10 20 30 40 50 60 70   (root = 40)
After deleting 40: 10 20 30 50 60 70   (root = 50)
After deleting 10: 20 30 50 60 70   (root = 50)
After deleting 60: 20 30 50 70   (root = 50)
25 is not in the tree: nothing to delete
After deleting 25: 20 30 50 70   (root = 50)
```

The time complexity is $O(h)$. The search walks one path down from the root, the successor search continues down from the node it found, and relinking is a constant number of pointer changes, so at most $h+1$ nodes are visited. This is $O(\log n)$ when the tree is balanced and $O(n)$ when it is a chain. The function uses $O(1)$ extra memory because it loops instead of recursing.

Annex: Drawings and Screenshots

Case 1

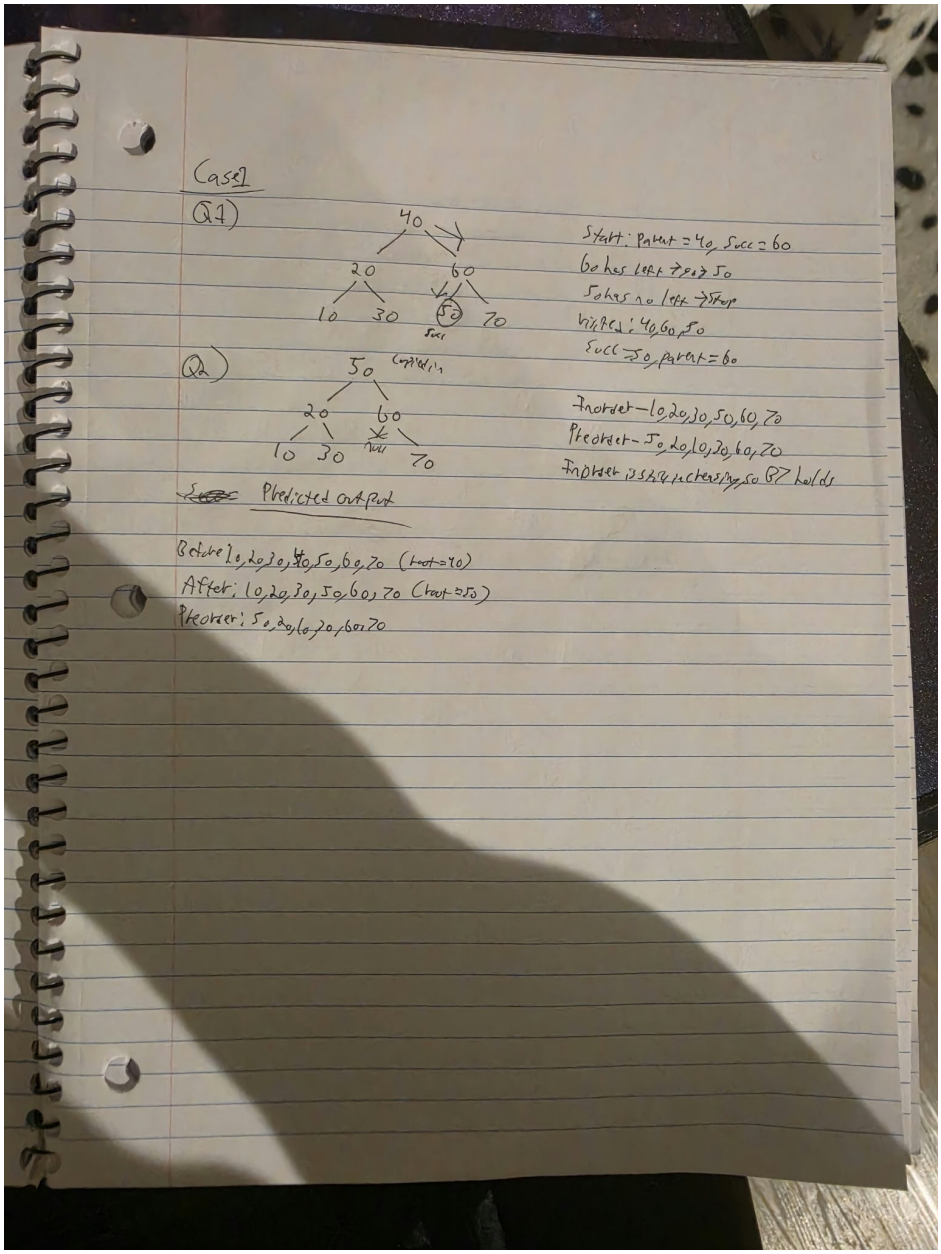


Figure 1: Case 1 drawings, Questions 1 and 2.

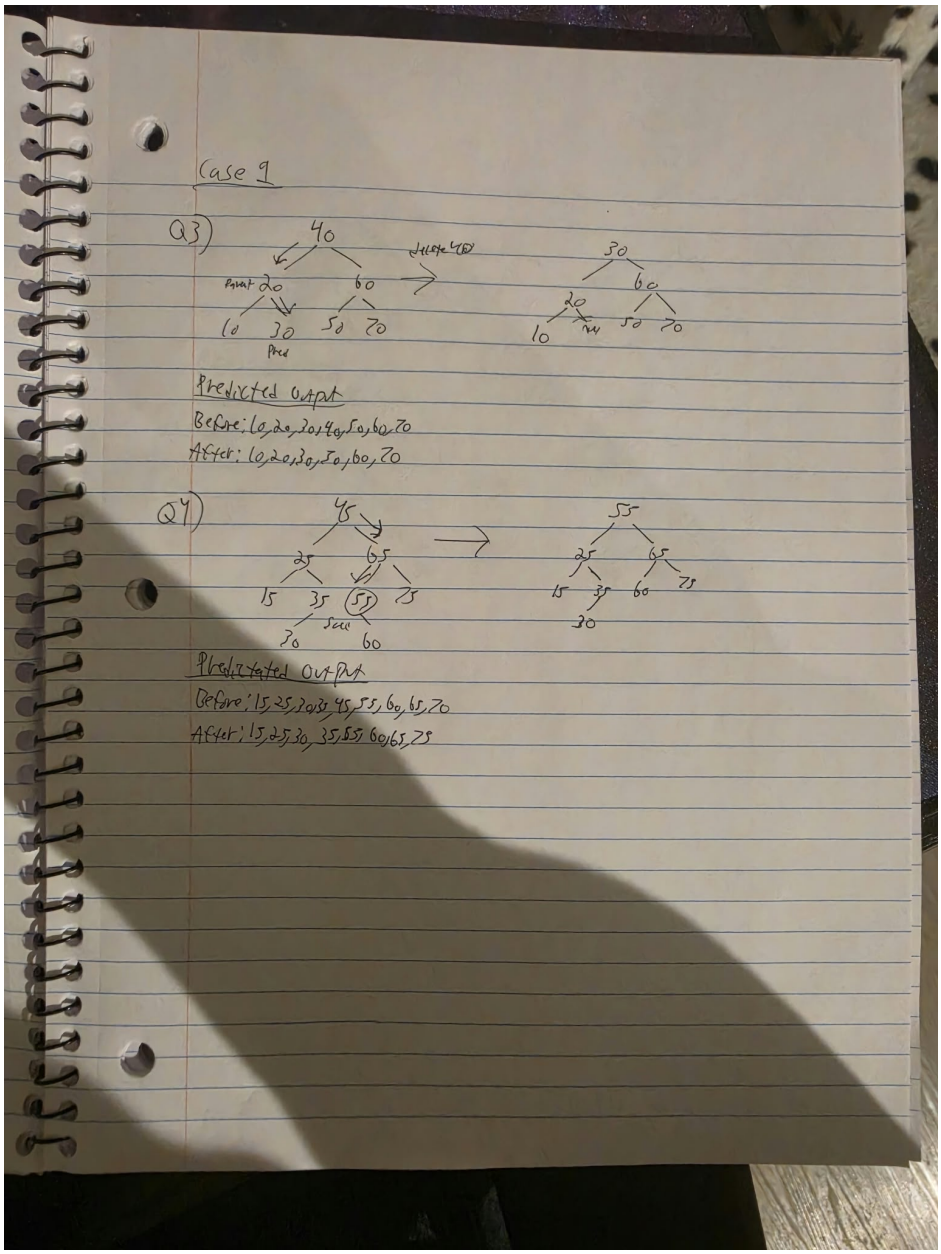


Figure 2: Case 1 drawings, Questions 3 and 4.

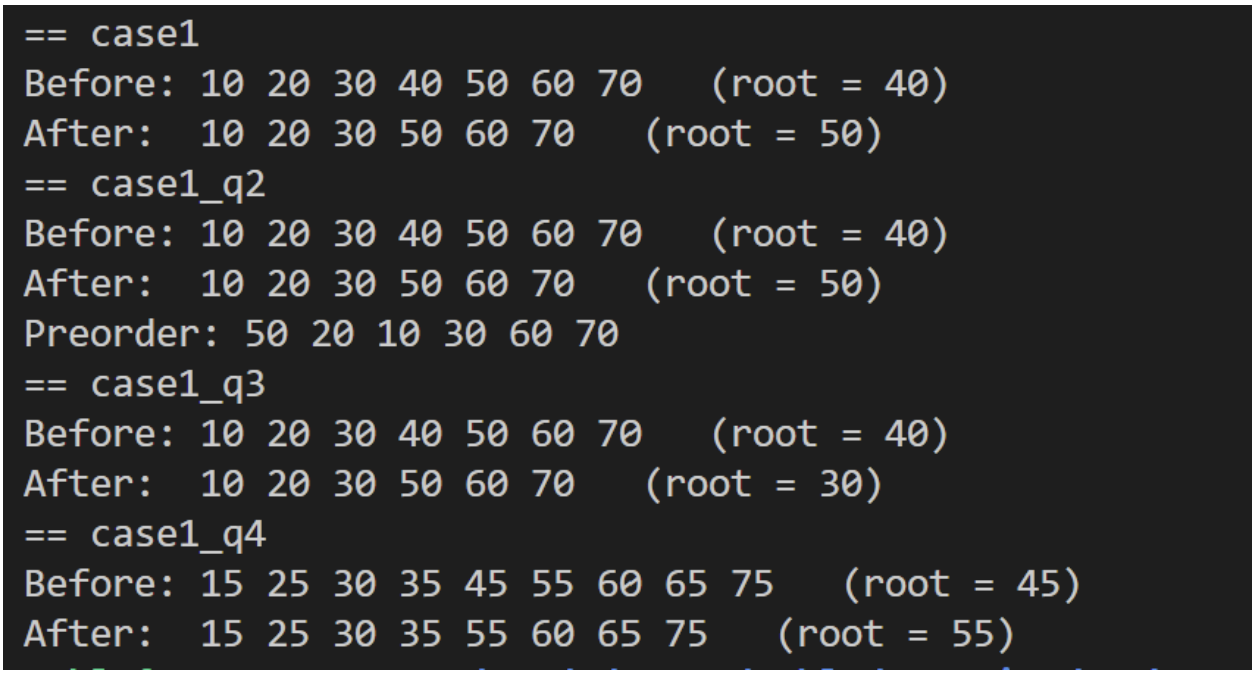


Figure 3: Case 1 runs: `case1`, `case1_q2`, `case1_q3`, `case1_q4`.

Case 2

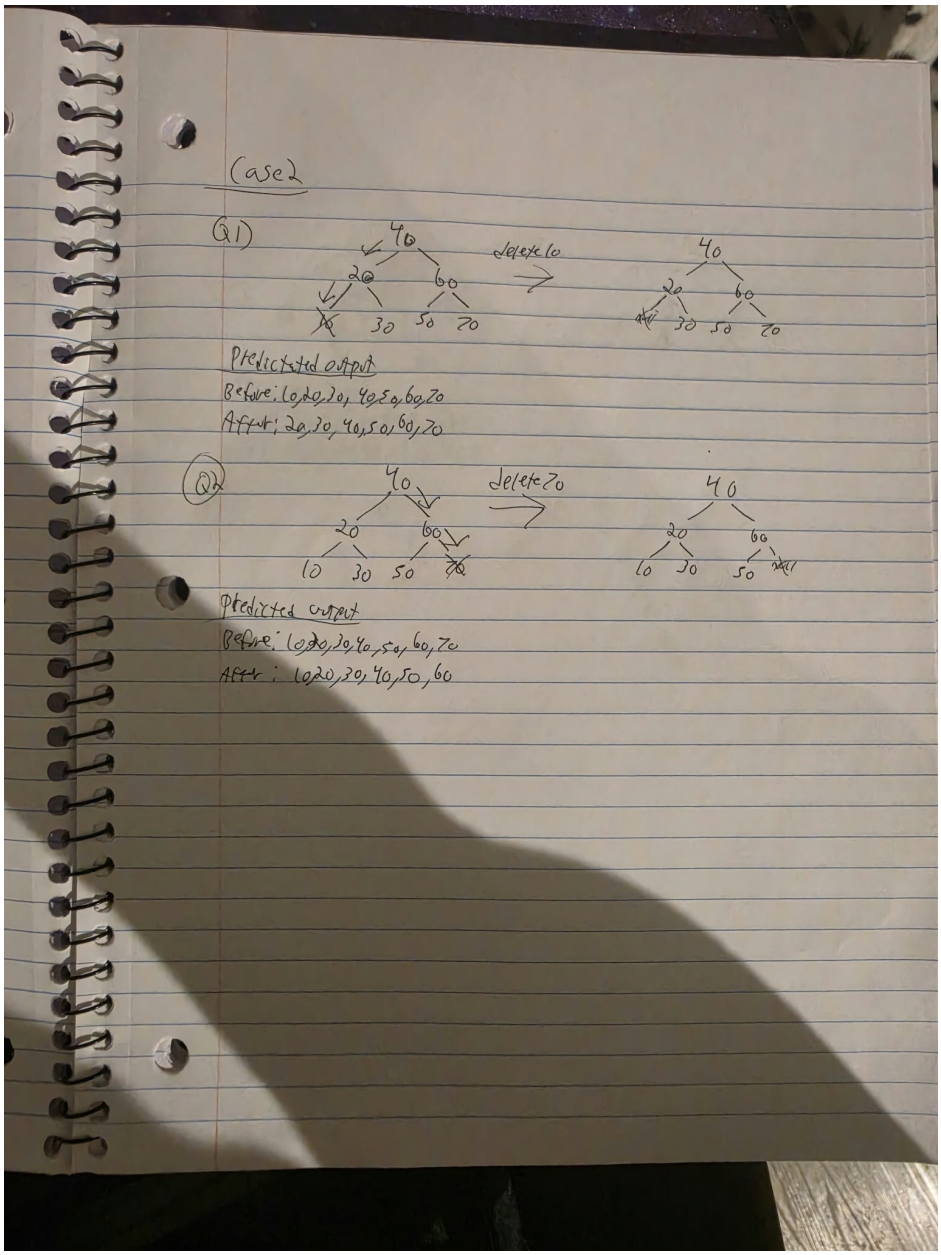


Figure 4: Case 2 drawings, Questions 1 and 2.

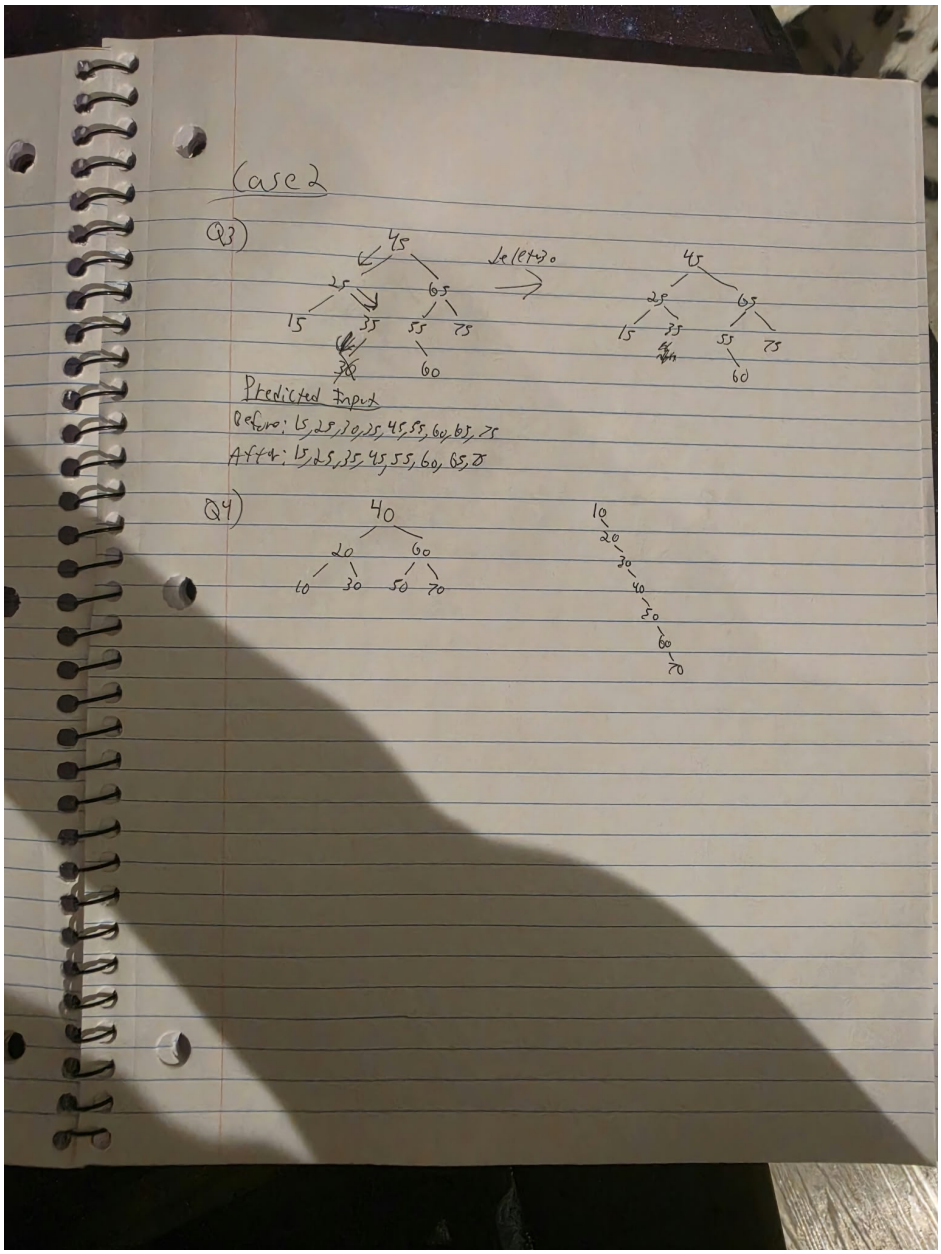


Figure 5: Case 2 drawings, Questions 3 and 4.

```
== case2
Before: 10 20 30 40 50 60 70   (root = 40)
After:  20 30 40 50 60 70   (root = 40)
== case2_q2
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 40 50 60   (root = 40)
== case2_q3
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 25 35 45 55 60 65 75   (root = 45)
```

Figure 6: Case 2 runs: `case2`, `case2_q2`, `case2_q3`.

Case 3

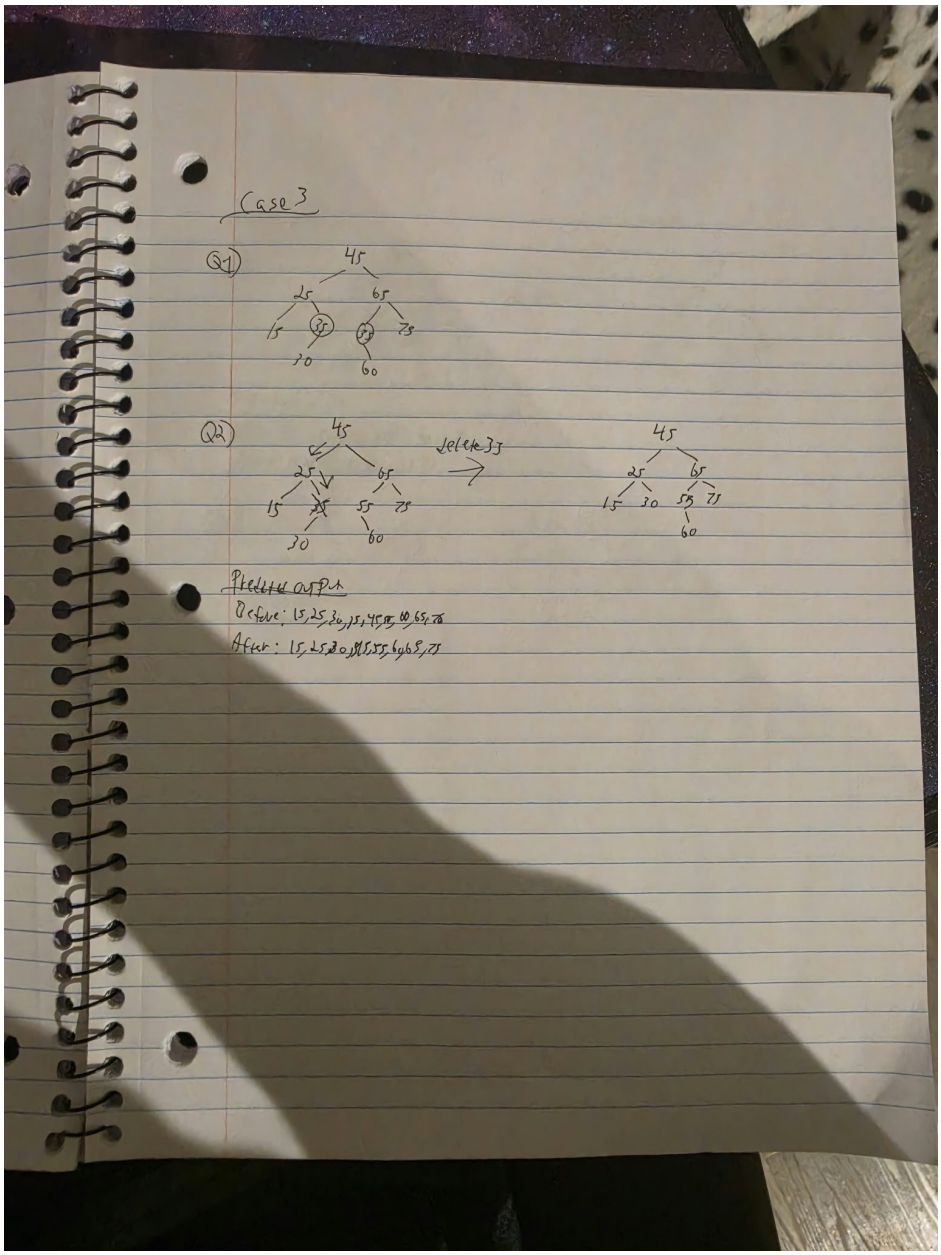


Figure 7: Case 3 drawings, Questions 1 and 2.

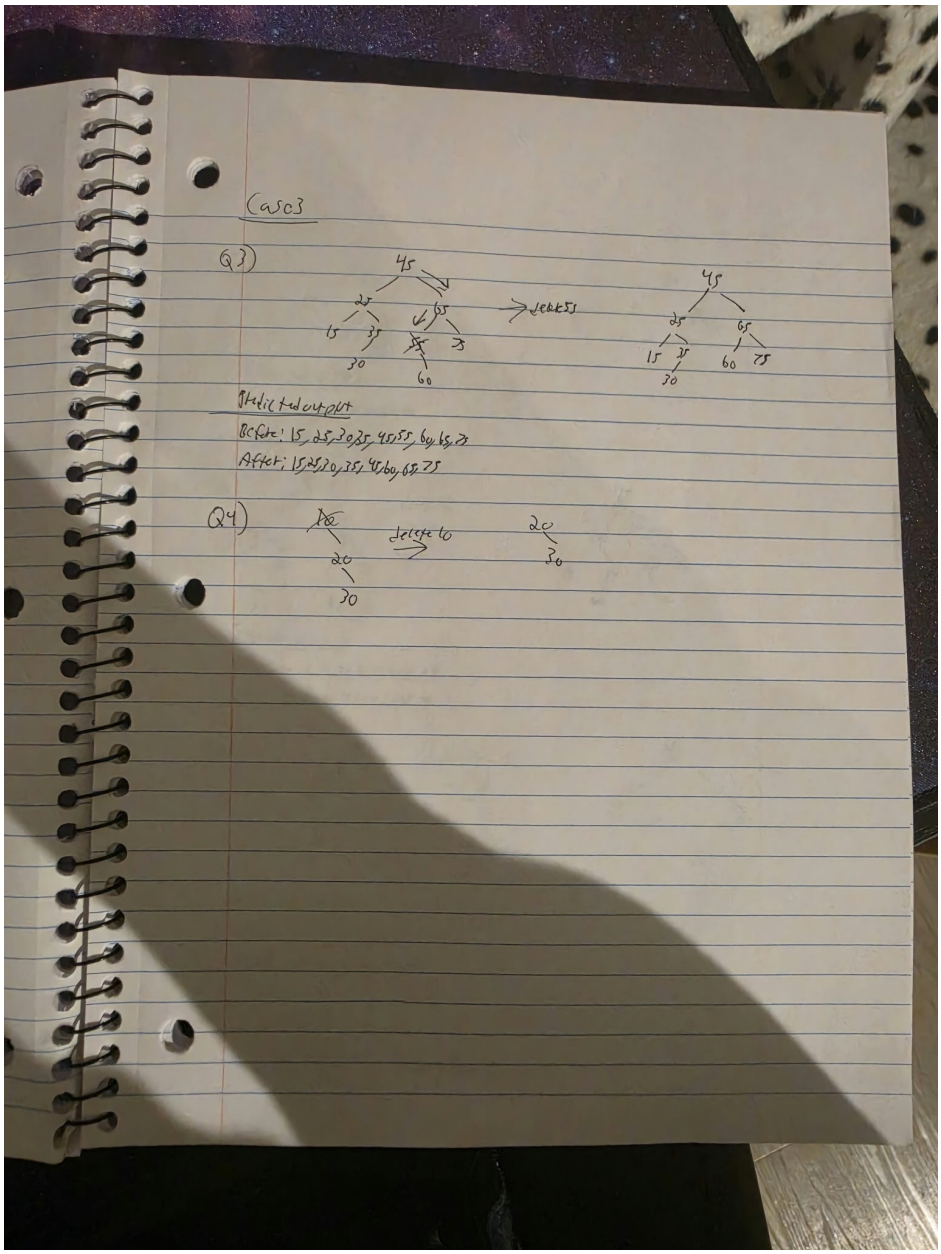


Figure 8: Case 3 drawings, Questions 3 and 4.

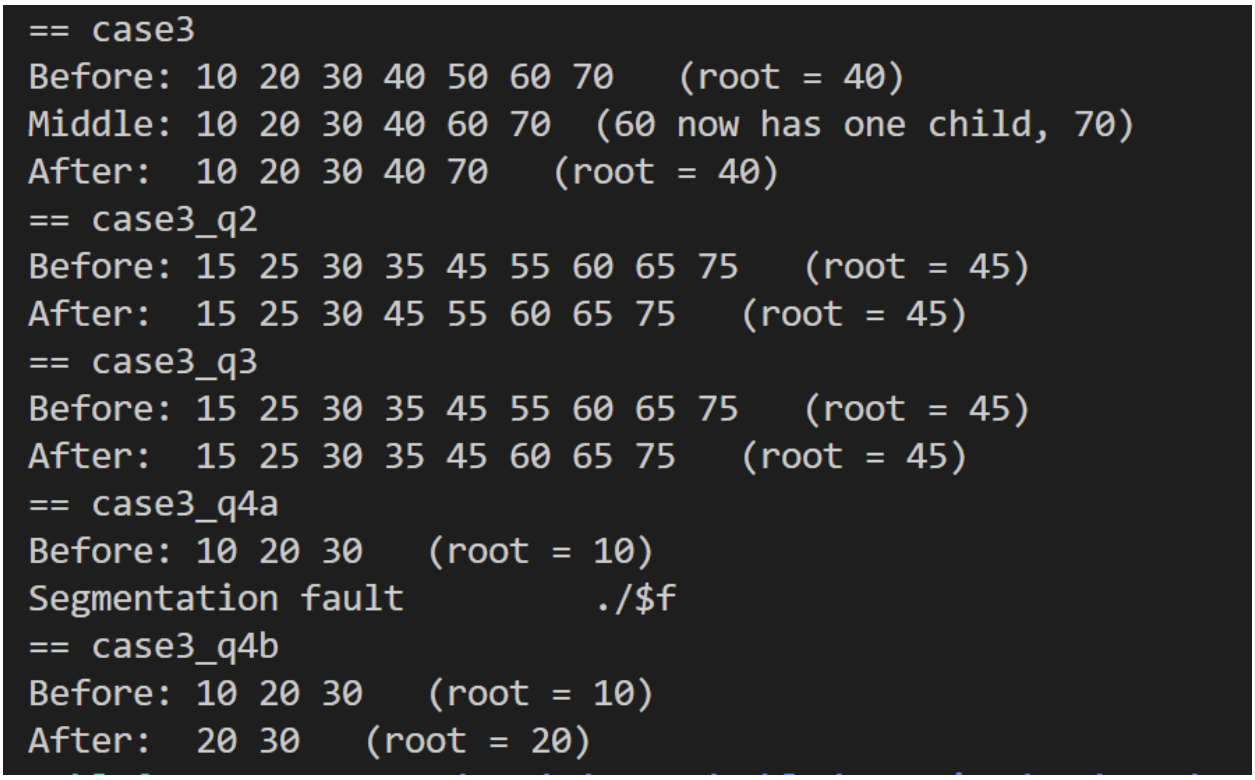


Figure 9: Case 3 runs: `case3`, `case3_q2`, `case3_q3`, `case3_q4a`, `case3_q4b`.

Case 4

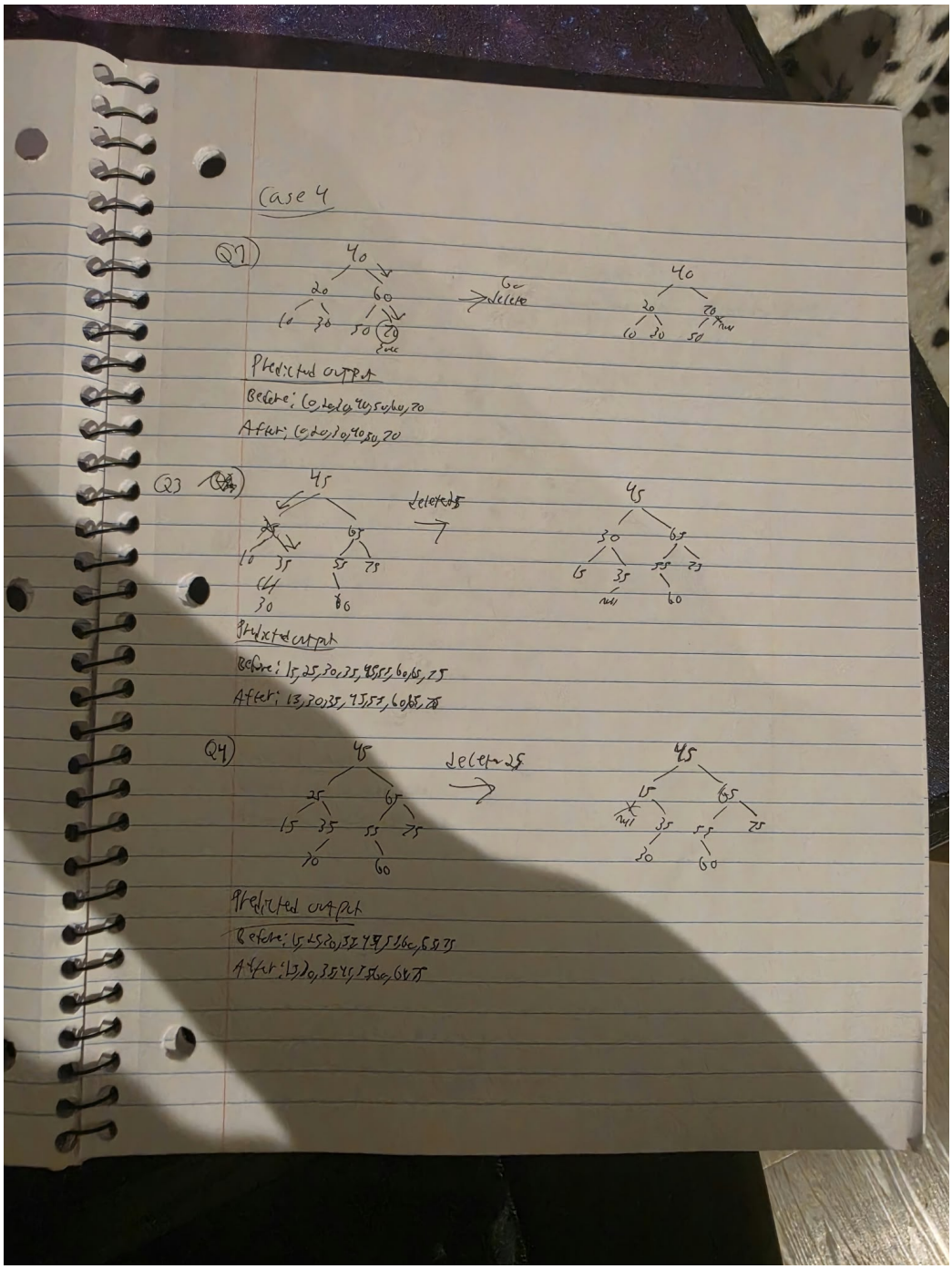


Figure 10: Case 4 drawings.

```
== case4
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 30 40 50 60 70   (root = 40)
== case4_q1
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 40 50 70   (root = 40)
== case4_q3
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 30 35 45 55 60 65 75   (root = 45)
== case4_q4
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
After:  15 30 35 45 55 60 65 75   (root = 45)
```

Figure 11: Case 4 runs: `case4`, `case4_q1`, `case4_q3`, `case4_q4`.

Case 5

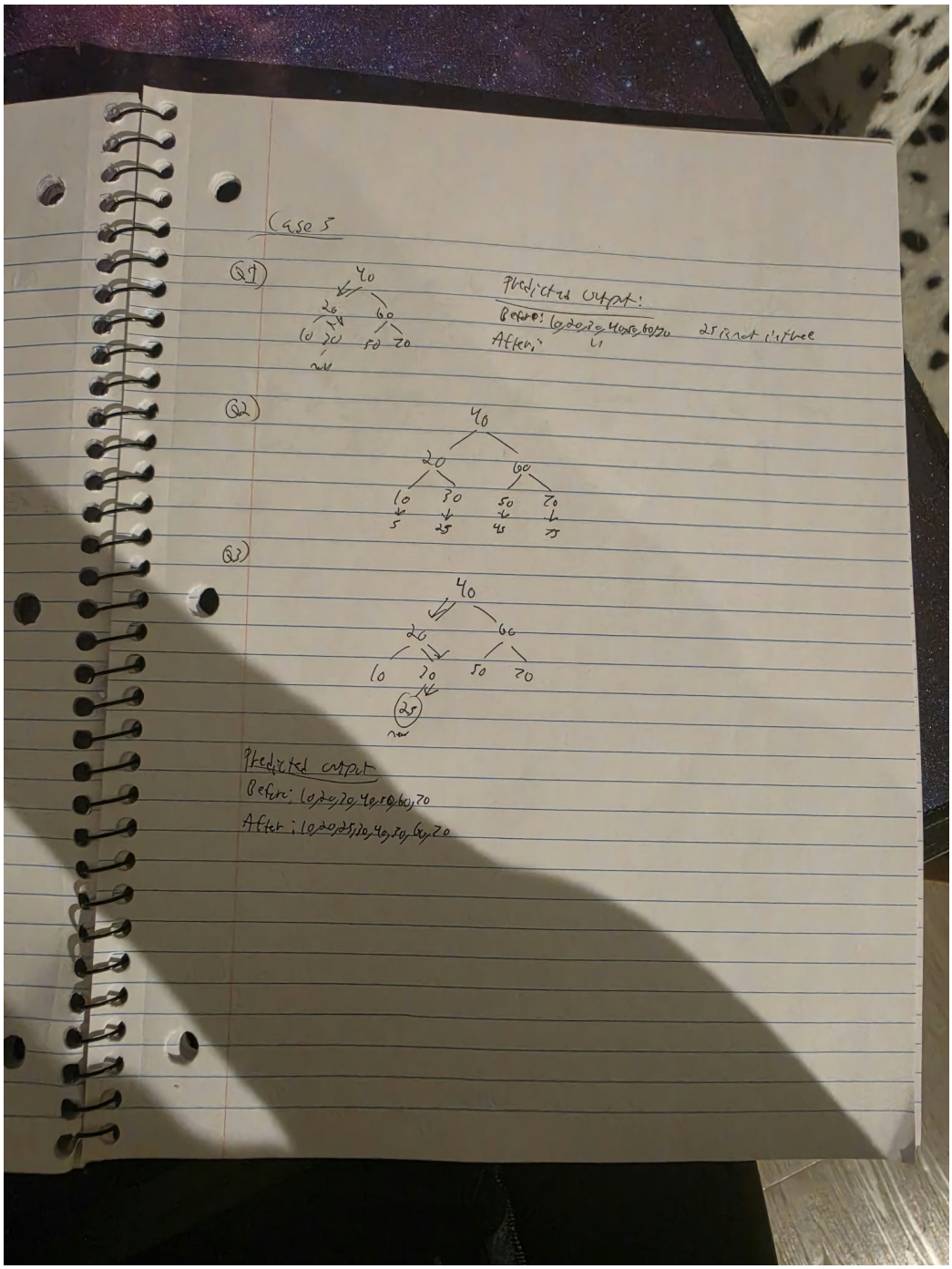


Figure 12: Case 5 drawings.

```
== case5
Before: 10 20 30 40 50 60 70   (root = 40)
25 is not in the tree: nothing to delete
After:  10 20 30 40 50 60 70   (root = 40)
== case5_q2
Before: 10 20 30 40 50 60 70   (root = 40)
25: 3 comparisons, stopped at 30->left
5: 3 comparisons, stopped at 10->left
45: 3 comparisons, stopped at 50->left
75: 3 comparisons, stopped at 70->right
After:  10 20 30 40 50 60 70   (root = 40)
== case5_q3
Before: 10 20 30 40 50 60 70   (root = 40)
25 was not in the tree: inserted it
After:  10 20 25 30 40 50 60 70   (root = 40)
== case5_q4a
Segmentation fault         ./$f
== case5_q4b
Before: the tree is empty
25 is not in the tree: nothing to delete
After:  the tree is empty
```

Figure 13: Case 5 runs: `case5`, `case5_q2`, `case5_q3`, `case5_q4a`, `case5_q4b`.

Bonus

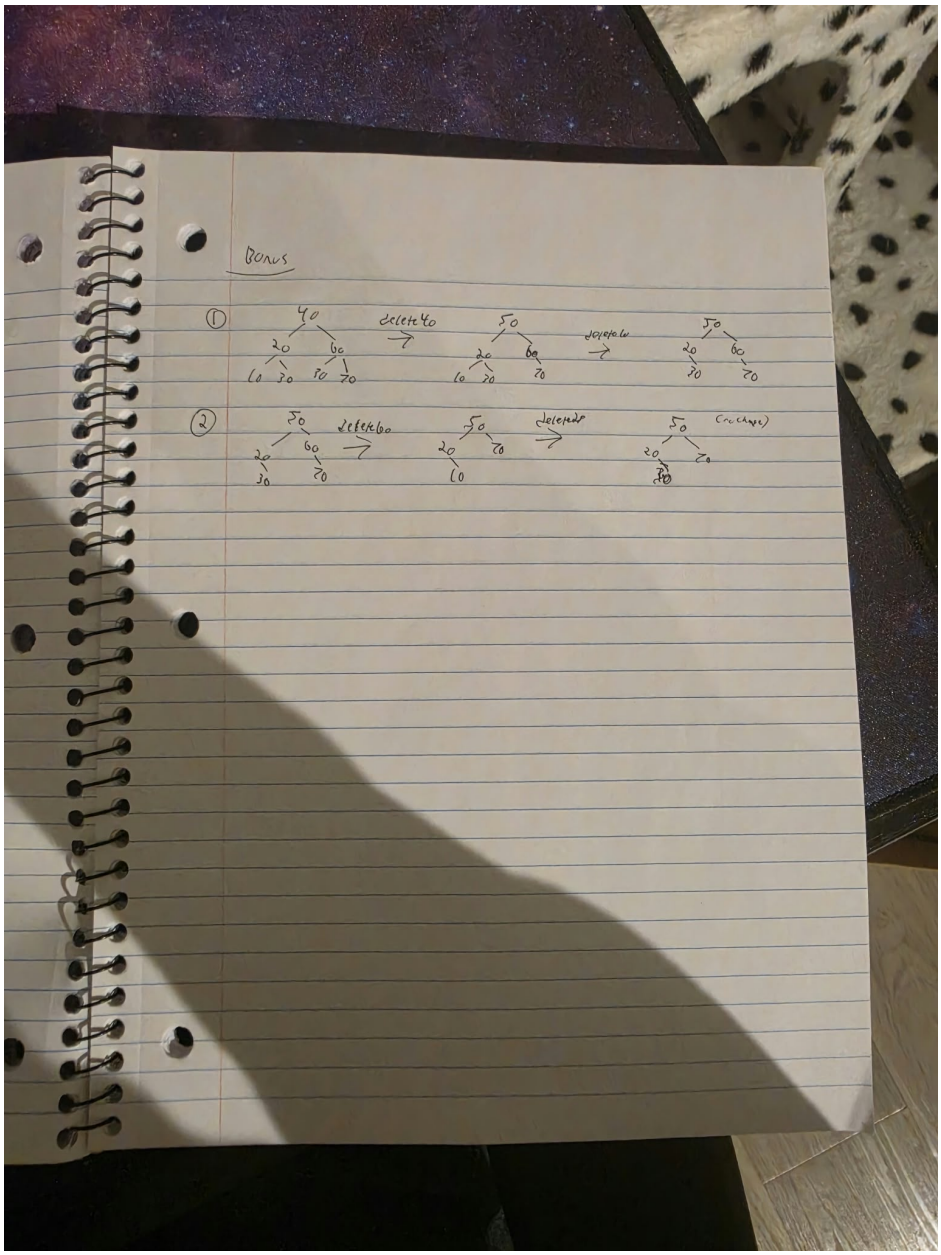


Figure 14: Bonus drawing: tree T after each deletion.

```
Before:              10 20 30 40 50 60 70   (root = 40)
After deleting 40: 10 20 30 50 60 70   (root = 50)
After deleting 10: 20 30 50 60 70   (root = 50)
After deleting 60: 20 30 50 70   (root = 50)
25 is not in the tree: nothing to delete
After deleting 25: 20 30 50 70   (root = 50)
```

Figure 15: Bonus run: `bonus`.